



PIC

Grado de Inteligencia Artificial  
Universidad de Vigo

## **ENTREGABLE 2: Sistema AIoT**

### **Inteligencia y Seguridad sobre un Pipeline IoT**

**Autores:** Raúl Vázquez Arévalo,  
Damián González Sarmiento,  
Lucas Alonso Alonso

Convocatoria junio / Curso 2025/2026

# 1. INTRODUCCIÓN Y DECISIONES DE ALTO NIVEL

## 1.1. Vía elegida y justificación

Para hacer esta segunda entrega, hemos decidido desarrollar la **Vía B**, la cual propone trabajar sobre la base del simulador de datos de telemetría industrial proporcionado, en lugar de extender el simulador propio desarrollado en la Entrega 1 (Vía A). Esta decisión se ha fundamentado en dos criterios clave:

- **Análisis de Coste-Beneficio y Riesgo:** Al evaluar los requisitos de la práctica, observamos que la Vía A implicaba un esfuerzo de desarrollo mayor sin que esto se tradujera en un incentivo de puntuación superior. Por ello decidimos la vía B.
- **Reutilización de conocimiento de dominio:** La Vía B trabaja sobre el mismo *dataset* y dominio físico del que se nos evaluó en el examen reciente y en sesiones prácticas anteriores (como la P9). Conocer el comportamiento de estos datos de antemano nos permitió recortar drásticamente el tiempo necesario para entender el *dataset* y centrar el esfuerzo en el análisis.

## 1.2 Dominio de aplicación

El proyecto se enmarca en el “Mantenimiento Predictivo Industrial”, monitorizando en tiempo real la salud de un motor mediante tres sensores: vibración, temperatura y corriente. El gran reto no es detectar anomalías obvias, sino afrontar dos fenómenos que inutilizan los sistemas de alertas tradicionales:

- **Concept Drift:** El motor sufre desgastes paulatinos y estacionales que alteran lentamente la línea base de los sensores, exigiendo modelos dinámicos que se adapten a una "nueva normalidad".
- **Anomalías Colectivas Multivariantes:** Las degradaciones tempranas no disparan un único sensor en bruto. El fallo real se manifiesta cuando se rompe la relación física entre variables

## 1.3. Arquitectura General del Sistema

Se ha desplegado una arquitectura IoT en los siguientes bloques:

1. **Bloque de Infraestructura Base y Simulación:** Partimos de la arquitectura inicial orquestada mediante `docker-compose.yml` y contenedores personalizados (`Dockerfile.ingestor`, `Dockerfile.jupyter`). Este bloque levanta el *broker* MQTT (Mosquitto), la base de datos temporal (InfluxDB) y la monitorización base (Grafana), sirviendo como ecosistema para el generador de telemetría (`simulador_e2.py`).
2. **Bloque de Detección Continua en el Edge (Tarea 1):** Concentrado en la modificación profunda del *script* `cliente_e2_skeleton.py`. Actúa como un nodo *Edge* que se suscribe al flujo MQTT, ejecuta la extracción de *features* y evalúa nuestro modelo de Ensamble en tiempo real, volcando automáticamente los registros de anomalías en el archivo `detecciones_e2.csv`.

3. **Bloque de Evaluación y Auditoría (Tarea 1):** Creación de herramientas propias (fuera del *pipeline* de producción) para auditar el rendimiento algorítmico. Incluye `metricas.py` para el cálculo analítico de falsos positivos y latencias, y `plot_comparativa.py` para generar visualmente las gráficas de comportamiento (`comparativa_4_estrategias.png`).
  4. **Bloque de Modelado Predictivo y Ciberseguridad (Tareas 2, 3 y 4):** Centralizado en el entorno interactivo `Demostracion_T2_T3_T4.ipynb`, el cual consolida las demostraciones empíricas de preprocesamiento, inferencia y *Threat Modeling*. Este entorno se apoya en el histórico `motor_historical.csv` para el entrenamiento, y utiliza el directorio `models/` como repositorio para almacenar y cargar ágilmente los modelos exportados (`.pkl`, `.onnx`, `.tf`), agilizando el proceso de validación y la inyección de ataques.
- 

Para la demostración empírica de la siguiente T1, hemos implementado el *pipeline* de detección en tiempo real dentro del script `cliente_e2_skeleton.py`. Este cliente se conecta al flujo MQTT del simulador, procesa la telemetría, evalúa simultáneamente las cuatro estrategias diseñadas (tres *baselines* y nuestra propuesta de Ensemble) y genera automáticamente un archivo de registro (`detecciones_e2.csv`). El análisis de estos resultados y las gráficas que se exponen a continuación se extraen mediante nuestros propios *scripts* de auditoría.

Para reproducir la evaluación en vivo de la Tarea 1:

```
``bash
```

```
docker-compose up -d -> levantar la infraestructura
```

```
python simulador_e2.py --fast -> lanzar el simulador de telemetría (en una terminal)
```

```
python cliente_e2_skeleton.py -> lanzar el detector de anomalías (en otra terminal)
```

```
**Esperar a que se genere el archivo 'detecciones_e2.csv' **
```

```
pip install paho-mqtt numpy scikit-learn joblib pandas tabulate -> instalar librerías necesarias
```

```
python metricas.py -> evalúa tabla de Falsos Positivos y Latencia
```

```
python plot_comparativa.py -> Genera la imagen con las gráficas de las 4 estrategias
```

```
``
```

## T1 - DETECCIÓN DE ANOMALÍAS ROBUSTA AL DRIFT

Para construir un detector capaz de adaptarse al *Concept Drift* del motor, la idea no ha sido únicamente buscar el algoritmo ganador, sino demostrar criterio como ingenieros comparando alternativas. Por ello, hemos estructurado el análisis evaluando el comportamiento de tres modelos Baseline (Umbral Fijo, Z-Score Adaptativo e Isolation Forest Estático) a lo largo de las tres fases del simulador.

El estudio de estos baselines nos ha permitido identificar sus limitaciones matemáticas frente al *drift* y nos ha proporcionado el contexto necesario para justificar la elección de nuestra **Propuesta Final**: una estrategia de adaptación basada en un Ensamble.

### Estrategia 1: Umbral fijo estático

```
ratio_actual = features.get("ratio_vib_curr", 0)
```

```
# 1. BASELINE 1: UMBRAL FIJO
resultados["score_fixed"] = float(ratio_actual)
if ratio_actual > modelo["fixed_ratio_threshold"]:
    resultados["anomaly_fixed"] = True
```

**Qué hace.** Evalúa una característica cruzada extraída de la ventana temporal actual. Si el valor supera el límite predefinido de 0.7, el sistema declara anomalía.

**Por qué `ratio_vib_curr` y no el valor bruto.** Utilizar la vibración en bruto es peligroso: un pico puede deberse a un aumento en la carga de trabajo. Sin embargo, el ratio vibración/corriente mide la ineficiencia mecánica. Si la vibración sube pero el consumo eléctrico se mantiene, el motor transforma la energía en temblores destructivos. Elegir una característica con significado físico es vital para evitar falsas alarmas operativas.

**Por qué 0.7.** Observando la fase de funcionamiento en la fase "normal" del simulador, este ratio oscila alrededor de 0.60. Establecer un umbral estático en 0.7 deja un margen de seguridad estadístico para absorber el ruido normal de los sensores sin disparar alarmas, quedando por encima de la media sana.

**Limitaciones propias y por qué falla.** Al estar calibrado sobre la fase "Normal", el umbral no sabe nada sobre las demás fases. En la fase de degradación, el *baseline* del ratio sube de forma natural y cruza la barrera del 0.7. Desde ese momento, dispara alertas el 100% del tiempo de forma prematura. Se convierte en un detector de "*el motor está más desgastado que ayer*", que es una afirmación verdadera pero inútil, mostrando su sensibilidad al drift.

**Trade-offs.** Sacrificamos adaptabilidad a cambio de una extrema eficiencia computacional. La huella de memoria y el consumo energético son críticos en dispositivos IoT de campo. Este detector tiene una complejidad de  $O(1)$  y solo requiere una comparación booleana, permitiendo su despliegue en microcontroladores minúsculos donde modelos avanzados como un *Isolation Forest* jamás cabrían.

**Propuestas de mejora.** Para hacer operable este enfoque en un entorno real, proponemos dos mejoras:

- **Relegar su función:** No utilizarlo como detector temprano predictivo, sino como un Límite de Seguridad Catastrófico de última línea.
- **Evolución dinámica:** Transformar este concepto en un umbral estadístico móvil que acompañe al *drift* del motor, lo cual da pie a la segunda estrategia analizada: el Z-Score Adaptativo.

## Estrategia 2: Z-score adaptativo

```
if len(historial) > 5: # Necesitamos un mínimo de datos para la std
    # Sacamos el historial reciente del ratio
    ratios_pasados = [h["features"].get("ratio_vib_curr", 0) for h in historial[-modelo["window_size"]:]]
    media_movil = np.mean(ratios_pasados)
    std_movil = np.std(ratios_pasados)

    if std_movil > 0:
        z_score = abs(ratio_actual - media_movil) / std_movil
        resultados["score_zscore"] = float(z_score)
        if z_score > modelo["z_threshold"]:
            resultados["anomaly_zscore"] = True
        else:
            resultados["score_zscore"] = 0.0
```

**Qué hace.** Calcula la media móvil ( $\mu$ ) y la desviación típica ( $\sigma$ ) del ratio de ineficiencia sobre las últimas 100 ventanas. Declara anomalía si la ventana actual se aleja más de  $3\sigma$  de esa referencia móvil.

**Por qué se adapta.** A diferencia del umbral fijo, la referencia de "lo normal" aquí es móvil. Observando la gráfica generada (véase en el código o en las siguientes páginas), vemos que al inicio de la fase de degradación (amarilla) el *score* sufre un pico abrupto disparando alertas, ya que el ratio cambia bruscamente respecto a la normalidad previa. Sin embargo, a medida que la ventana avanza, la media histórica asimila estos nuevos valores altos y la desviación típica se ensancha. Como resultado, la curva verde desciende paulatinamente por debajo del umbral de 3.0, "acostumbrándose" al estado de degradación.

**Limitaciones propias y por qué falla.** Al adaptarse continuamente al Concept Drift, el modelo sufre de amnesia respecto al estado sano original del motor. Como se observa en la zona roja de la gráfica (Fallo), el Z-Score no lanza ninguna alerta durante la rotura crítica. Esto ocurre porque la media móvil y la desviación típica se han ido adaptando a la rampa de degradación previa, ensanchando la tolerancia del algoritmo hasta el punto de asimilar los niveles de ineficiencia de la rotura como parte de su "nueva normalidad". Para cuando llega el fallo, el detector considera que tener una ineficiencia altísima es "lo normal por definición". Es el equivalente a un ventilador que pierde eficiencia durante semanas y nunca dispara la alarma, falla en el momento clave.

**Trade-offs.** La decisión de usar un Z-Score adaptativo implica un *trade-off* severo: sacrificamos la detección de fallos por desgaste a cambio de eliminar los Falsos Positivos causados por derivas (como cambios de temperatura ambiente día/noche).

**Propuestas de mejora.** Para hacer que un modelo basado en Z-Score sea operable en este contexto, se proponen dos mejoras de diseño:

- **Cambio de Feature:** Como se vio en prácticas anteriores, aplicar el Z-Score sobre la media diluye los picos cortos. Si se aplicase sobre la pendiente o la varianza de la ventana en lugar de la media, el modelo sería mucho más reactivo a las fluctuaciones erráticas previas a la rotura.

- **Arquitectura de Doble Ventana:** En lugar de una sola ventana móvil, utilizar un sistema que compare una media a corto plazo contra una media a muy largo plazo "anclada". Esto permitiría absorber el ruido diario pero dispararía la alerta cuando la deriva acumulada se alejase irremediabilmente del estado original de fábrica.

### Estrategia 3: Isolation Forest

```
if modelo["is_fitted"]:  
    prediccion = modelo["iforest"].predict(x_val)[0]  
    score_iforest = -1.0 * modelo["iforest"].score_samples(x_val)[0]  
    resultados["score_iforest"] = float(score_iforest)  
    resultados["anomaly_iforest"] = bool(prediccion == -1)  
  
resultados["anomaly"] = resultados["anomaly_iforest"]  
resultados["score"] = resultados["score_iforest"]
```

**Qué hace.** Entrena un IsolationForest con las ventanas del Tramo A, todas etiquetadas implícitamente como "normal". El modelo aprende la forma del espacio de features en ese régimen estable de la normalidad del tramo. Cualquier punto que quede fuera de esa región recibe puntuación de anomalía.

**Conexión con contamination=0.05.** Se configuró con una contaminación del 0.05 para evitar que el modelo no sea ni demasiado restrictivo ni demasiado tolerante, haciendo que IF asuma un 5% de ruido estadístico durante los datos de entrenamiento en régimen normal. En el primer tramo no se lanzan las anomalías correspondientes al porcentaje de la contamination ya que le decimos al algoritmo que hasta que no se acabe de entrenar (entrenamiento en la fase "Normal") no puede lanzar alarmas (detectar anomalías).

**Limitaciones:** Durante la evaluación, se observó que el modelo estático genera una tasa inaceptable de falsos positivos al adentrarse en la fase de degradación. Al procesar registros de ineficiencia mecánica ausentes en su distribución de entrenamiento original, el algoritmo experimenta una severa *Model Degradation* a causa del *Data Drift*.

En fases iniciales del análisis se evaluó la viabilidad de implementar un reentrenamiento periódico mediante ventanas deslizantes para mitigar este efecto. No obstante, los resultados evidenciaron que el *Isolation Forest* presenta limitaciones estructurales para extrapolar tendencias monótonas. Dado que la degradación física del motor es un proceso acumulativo y constante, las nuevas observaciones se ubican sistemáticamente fuera de la frontera de decisión definida por la ventana de entrenamiento inmediatamente anterior, saturando el clasificador y manteniendo el sistema en un estado de alerta continuo.

**Trade-offs.** El uso del IF plantea el *trade-off* más severo en un entorno IoT: mayor capacidad analítica multivariante a costa de una fragilidad absoluta ante el paso del tiempo. Evaluar el *forward pass* de un bosque de 100 árboles de decisión exige una huella de memoria inviable para microcontroladores básicos de campo, lo que obligaría a trasladar este modelo a un nodo superior aumentando la latencia y la dependencia de red.

### Propuestas de mejora:

- **Elaborar un ensamble:** Utilizar z-score adaptativo como detector principal y IF como segunda opinión. Esta combinación la vamos a probar a continuación para intentar mejorar el rendimiento de nuestro detector.

### Estrategia 4: Ensamble (Z-Score junto IF)

```
# 3B. Detector Principal: Z-Score sobre el Máximo (Reactivo a fallos)
feature_max = features.get("vibration_max", 0)
z_score_max = 0.0
anomalia_z_max = False

if len(historial) > 5:
    historico_max = [h["features"].get("vibration_max", 0) for h in historial[-modelo["window_size"]:]]
    media_max = np.mean(historico_max)
    std_max = np.std(historico_max)

    if std_max > 0:
        z_score_max = abs(feature_max - media_max) / std_max
        anomalia_z_max = bool(z_score_max > modelo["z_threshold"])

# 3A. Segunda opinión: Isolation Forest Estático
keys = [k for k in sorted(features.keys()) if k not in ("timestamp", "time_rel", "phase", "resultado")]
x_val = np.array([features[k] for k in keys]).reshape(1, -1)

if len(historial) >= modelo["window_size"] and not modelo["is_fitted"]:
    X_train = np.array([[h["features"][k] for k in keys] for h in historial])
    modelo["iforest"].fit(X_train)
    modelo["is_fitted"] = True
    print(f"✅ IF Entrenado (Estático). Actuando como validador de contexto...")

anomalia_if_estatico = False
if modelo["is_fitted"]:
    prediccion = modelo["iforest"].predict(x_val)[0]
    anomalia_if_estatico = bool(prediccion == -1)
```

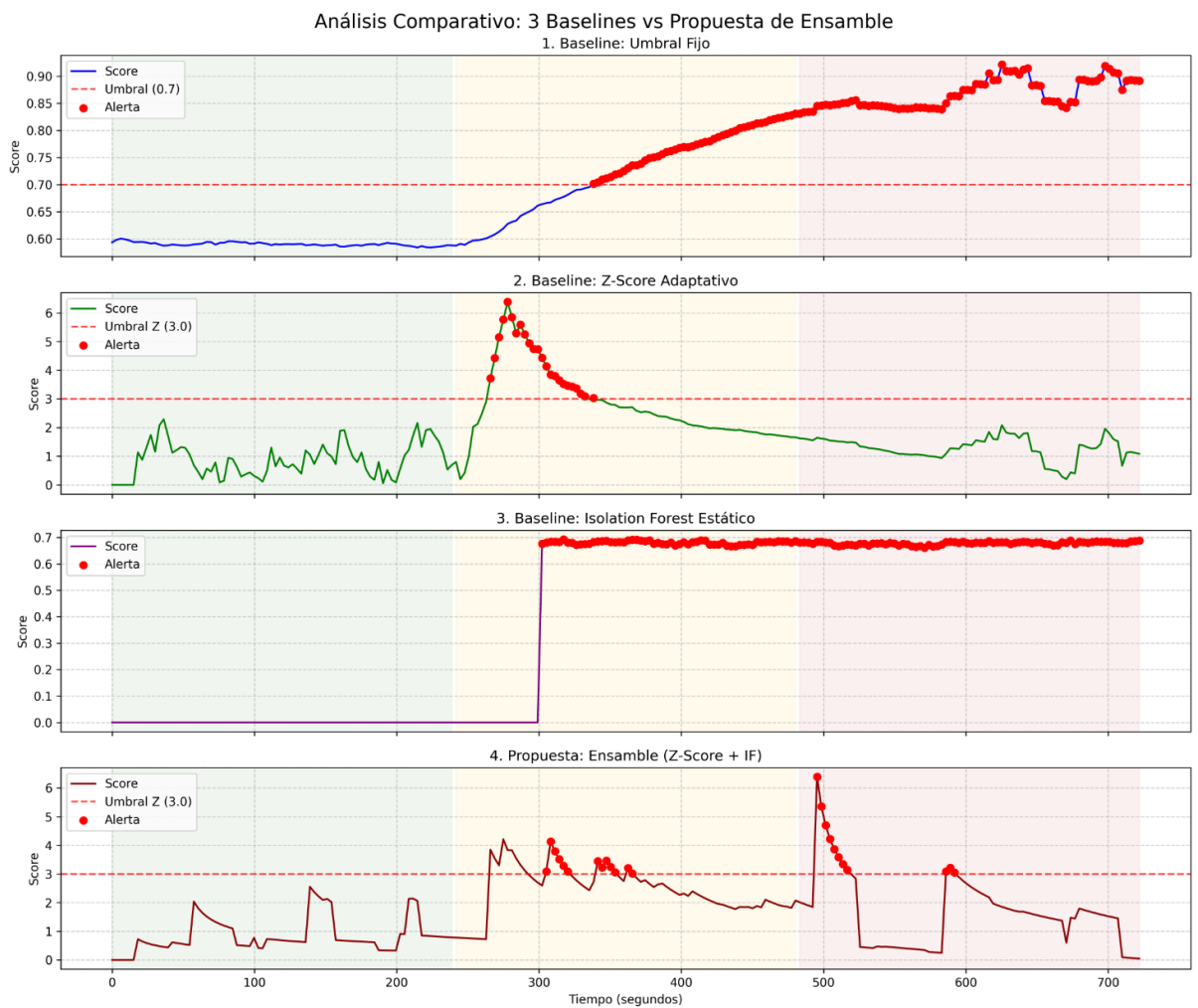
Atendiendo al requisito de implementar una estrategia explícita de adaptación, y tras analizar los baselines, descartamos el uso de un IF reentrenado (por su tendencia a la ceguera de extrapolación en derivas monótonas) y optamos por una arquitectura de Ensamble con validación de segunda opinión.

**Qué hace:** Utiliza un Z-Score Adaptativo configurado sobre el máximo de vibración como detector principal de picos (para evitar que el fallo se diluya en la media). Sin embargo, para que el Z-Score no pite por ruido en estado sano, se utiliza el Isolation Forest estático como validador de contexto mediante una compuerta lógica AND. El sistema solo alerta si hay un salto reactivo (Z-Score) y el motor ya ha salido del régimen seguro de fábrica (IF).

**Análisis Crítico de la Comparativa (Baselines vs Propuesta Final)**

Tras someter las cuatro estrategias al flujo de telemetría del simulador, los resultados consolidan el análisis teórico:

| Tramo (Fase) | Umbral Fijo                | Z-Score adaptativo         | IF estático                | Z-Score adaptativo junto IF |
|--------------|----------------------------|----------------------------|----------------------------|-----------------------------|
| Normal       | 0/80 (0%)<br>Lat: N/A      | 0/80 (0%)<br>Lat: N/A      | 0/80 (0%)<br>Lat: N/A      | 0/80 (0%)<br>Lat: N/A       |
| Degradación  | 48/80 (60%)<br>Lat: 96.6 s | 24/80 (30%)<br>Lat: 24.1 s | 60/80 (75%)<br>Lat: 60.5 s | 13/80 (16%)<br>Lat: 63.4 s  |
| Fallo        | 80/80 (100%)<br>Lat: 0.0 s | 0/80 (0%)<br>Lat: N/A      | 80/80 (100%)<br>Lat: 0.0 s | 11/80 (14%)<br>Lat: 12.1 s  |



Como se observa en la tabla de métricas y en la evolución temporal de la gráfica adjunta, cada aproximación Baseline presenta un fallo sistémico incompatible con la industria real:

- **Umbral Fijo:** Es computacionalmente simple, pero falla estrepitosamente cuando cambia el régimen. Al estar calibrado sobre la fase sana, cruza el límite de 0.7 de forma permanente durante la rotura (100% de alertas), lo cual provoca fatiga de alertas.
- **Z-Score Adaptativo (sobre la media):** Es barato y adaptable al *Concept Drift* (solo 30% de alertas en degradación), pero el precio de esta adaptación es que pierde sensibilidad a cambios lentos. Al recalcular su referencia, asimila la rotura final como "lo normal" y se queda completamente ciego en el peor momento (0% de alertas en la fase de fallo).
- **Isolation Forest Estático:** Funciona perfectamente al principio, pero sin reentrenamiento se rompe a la primera derivada. Sufre de *model degradation* puro provocado por el *data drift*. Dispara un 75% en degradación y un 100% en fallo simplemente por estar fuera de su rango de entrenamiento.
- **Nuestra Propuesta Final (Ensamble):** Siguiendo las directrices de diseño de sistemas operables de verdad, hemos combinado las fortalezas del Z-Score (reaccionando a picos usando el máximo de vibración) validado por el contexto multivariante del IF (sistema de votación). El resultado es el único perfil verdaderamente operable: limpia el ruido en régimen normal (0%), proporciona una alerta temprana controlada en degradación (16%) asimilando el *drift* sin fatiga, y caza con éxito el colapso mecánico aislando sus alertas en el fallo crítico (14%).

Con respecto al muestreo de las latencias, tenemos el siguiente análisis:

A simple vista, el Umbral Fijo y el IF Estático parecen tener una reacción instantánea (0.0s) en la fase de fallo. Sin embargo, esto es un artefacto estadístico producto de su *Model Degradation*: al estar disparando alertas casi continuamente durante la fase previa de degradación, entran a la fase de fallo ya saturados. No detectan el inicio del fallo, simplemente arrastran la alarma anterior.

Por el contrario, nuestro modelo de Ensamble demuestra el único perfil operable. Tras haber proporcionado una alerta temprana (16%) y haber asimilado la rampa de desgaste posterior silenciando las alertas, presenta una latencia real de 12.1 segundos ante la rotura crítica. Operativamente, esto significa que el sistema requiere apenas 4 ventanas de evaluación (a 3 segundos por intervalo) para que el impacto físico del fallo dispare de nuevo el detector de picos (Z-Score sobre el máximo), aislando el evento crítico con buena precisión temporal y sin el enmascaramiento propio de la fatiga de alertas. Es precisamente esta garantía de reacción rápida y verificable, libre del ruido de falsas alarmas, lo que justifica técnica y operativamente la elección del Ensamble como nuestra solución definitiva.

En conclusión, el Ensamble es nuestra solución definitiva que se fundamenta en su capacidad para neutralizar las debilidades individuales de cada algoritmo, logrando el Ensamble el equilibrio operativo perfecto. Al utilizar el IF estático exclusivamente como validador de contexto y el Z-Score sobre el máximo como detector reactivo, se obtiene un sistema robusto que asimila el desgaste progresivo de la máquina, pero garantiza una detección precisa e inmediata frente al colapso catastrófico.

## Limitaciones propias del Ensamble

Aunque el Ensamble diseñado logra el mejor perfil operativo de todos para la monitorización reactiva, como ingenieros debemos identificar sus **limitaciones estructurales propias**:

1. **Dependencia del Contexto Estático:** El ensamble utiliza el *Isolation Forest* estático como compuerta lógica. Si el motor sufriera un cambio de régimen legítimo a largo plazo (por ejemplo, una actualización de *firmware* que aumente su carga de trabajo base), el IF estático quedaría permanentemente saturado. En este escenario, el ensamble perdería su capa de validación de contexto y quedaría expuesto nuevamente al problema del Z-Score, donde la actualización continua de la media y el ensanchamiento de la varianza durante el desgaste progresivo provocan que el algoritmo asimile el colapso final como un valor estadísticamente normal.
2. **Naturaleza Reactiva, no Predictiva:** El sistema es capaz de detectar la rotura con 12.1 segundos de latencia, salvando la integridad catastrófica de la planta. Sin embargo, no proporciona una ventana de tiempo suficiente para la planificación logística (pedir piezas de repuesto o programar paradas de mantenimiento).

## Propuesta de Mejora del Ensamble

Para resolver esta barrera operativa, el sistema no puede depender exclusivamente de algoritmos de detección de anomalías no supervisados, por muy sofisticados o combinados que estén. La evolución natural y necesaria de esta arquitectura es transicionar hacia un modelo de Aprendizaje Supervisado. Esta limitación justifica directamente el desarrollo de la Tarea 2, donde sustituiremos la reacción a la anomalía por un modelo predictivo (Random Forest Regressor) capaz de estimar la Vida Útil Restante (RUL), pasando de un mantenimiento reactivo de urgencia a un verdadero mantenimiento predictivo a largo plazo.

---

Para los cálculos de las siguientes tareas hemos realizado los cálculos de manera empírica en el notebook "*Demostracion\_T2\_T3\_T4.ipynb*" localizado en el directorio raíz del proyecto, pero se discutirá en la memoria los diferentes resultados y gráficas visuales obtenidos de manera explicativa. El notebook ya viene cargado por defecto con todos los resultados y con el modelo utilizado para la T1 "*detector\_t1.pk*" para el problema ya cargado en la carpeta "*/models*" además de otros dos modelos que usaremos para las pruebas.

Para ejecutarlo:

```
```bash
# 1. Levantar la infraestructura (Mosquitto + InfluxDB + Grafana + Ingestor)
docker-compose up -d

# 6. Ejecutar el notebook "Demostracion_T2_T3_T4.ipynb"
# Abrir http://localhost:8888/?token=pic2026
# Contiene las demostraciones empíricas de las tareas T2, T3 y T4 que aparecen
explicadas en la memoria
```
```

## T2 - COMPONENTE PREDICTIVO

### Decisión de Diseño

Antes de nada, en base al problema, para saber cual de las dos opciones tomar, hemos visualizado el dataset de los motores:

Filas totales: 101,163  
Motores únicos: 500  
Columnas: ['motor\_id', 'cycle', 'vibration\_mean', 'vibration\_std', 'temperature\_mean', 'temperature\_slope', 'current\_mean', 'current\_std', 'RUL']

| motor_id | cycle | vibration_mean | vibration_std | temperature_mean | temperature_slope | current_mean | current_std | RUL    |     |
|----------|-------|----------------|---------------|------------------|-------------------|--------------|-------------|--------|-----|
| 0        | 1     | 1              | 2.745         | 0.2885           | 40.95             | 0.0133       | 4.320       | 0.1144 | 125 |
| 1        | 1     | 2              | 2.779         | 0.3320           | 41.26             | 0.0098       | 4.370       | 0.1039 | 125 |
| 2        | 1     | 3              | 2.512         | 0.3368           | 41.25             | -0.0028      | 4.313       | 0.1432 | 125 |
| 3        | 1     | 4              | 2.569         | 0.3050           | 42.08             | 0.0295       | 4.296       | 0.1651 | 125 |
| 4        | 1     | 5              | 2.333         | 0.3243           | 40.33             | 0.0251       | 4.458       | 0.1600 | 125 |

Por lo que, se ha optado por la **Opción A (Regresión para RUL)** frente a la Fusión Sensorial. La decisión se fundamenta en la siguiente evaluación arquitectónica:

| Criterio                  | Opción A: Regresión para RUL                                                  | Opción B: Fusión Bayesiana                                   |
|---------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------|
| Objetivo Principal        | Predecir cuántos ciclos de vida le quedan al motor antes del fallo.           | Estimar un estado oculto actual con sensores ruidosos.       |
| Valor de Negocio          | Permite planificar el mantenimiento preventivo y pedir piezas con antelación. | Útil para control en tiempo real, pero no proyecta a futuro. |
| Complejidad Computacional | Alta en entrenamiento, moderada en inferencia.                                | Alta en inferencia iterativa (ej. Filtro de Kalman).         |

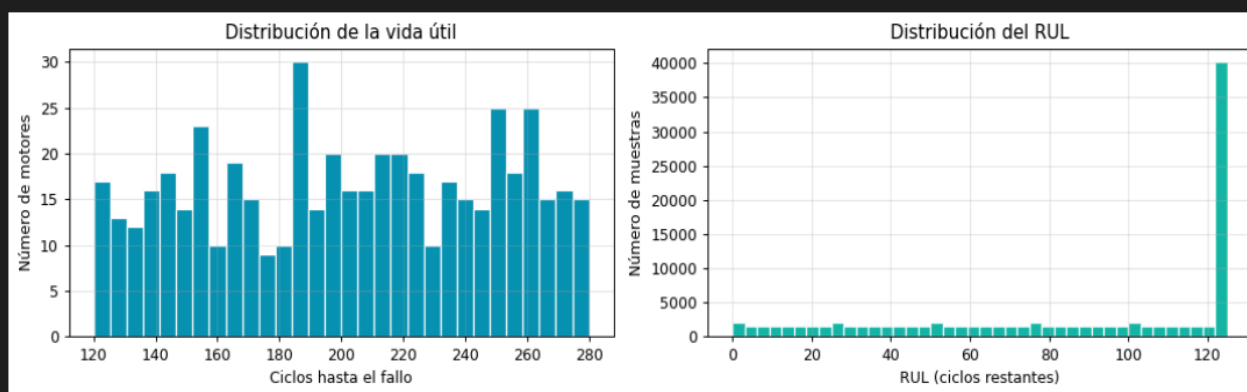
|                                  |                                                                                          |                                                                            |
|----------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Viabilidad en el Proyecto</b> | <b>Elegida.</b> Encaja perfectamente con la necesidad de predecir el desgaste del motor. | <b>Descartada.</b> No resuelve el problema de planificación a largo plazo. |
|----------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|

## Fuente de Datos

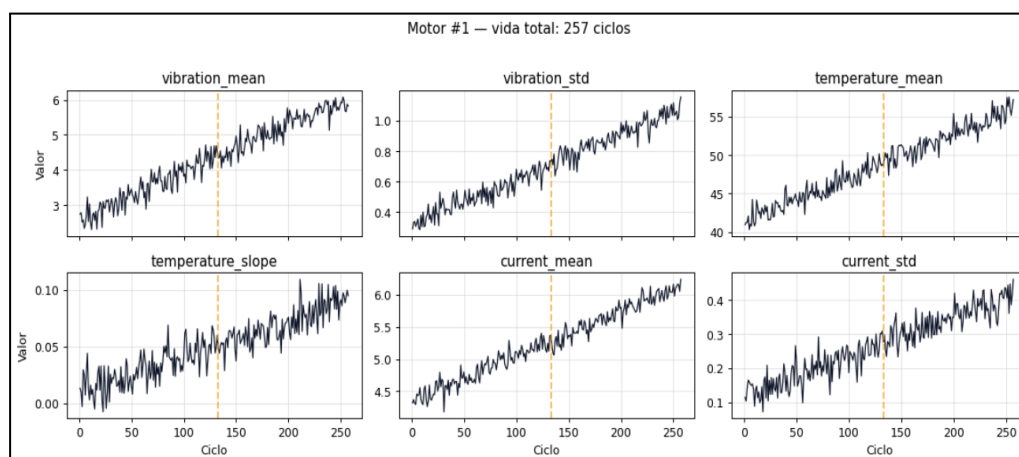
El entrenamiento se ha realizado utilizando el dataset proporcionado “*motor\_historical.csv*”, el cual contiene la telemetría completa de 500 motores desde su inicio hasta el fallo. Este dataset simula dinámicas de degradación mediante diferentes variables (deriva de temperatura, picos de vibración).

La variable objetivo (RUL) está limitada a un máximo de 125 ciclos. Esta decisión de diseño es coherente con la realidad industrial: predecir si a un motor le quedan 300 o 250 ciclos carece de valor operativo; el sistema solo necesita precisión cuando el componente entra en su ventana crítica de desgaste y como podemos observar en las gráficas, la gran mayoría de los motores tienen una vida mayor a 125, siendo la media en 202.3:

Vida media: 202.3 ciclos



Aquí analizamos visualmente cómo evolucionan las características de un motor a lo largo de su vida útil:



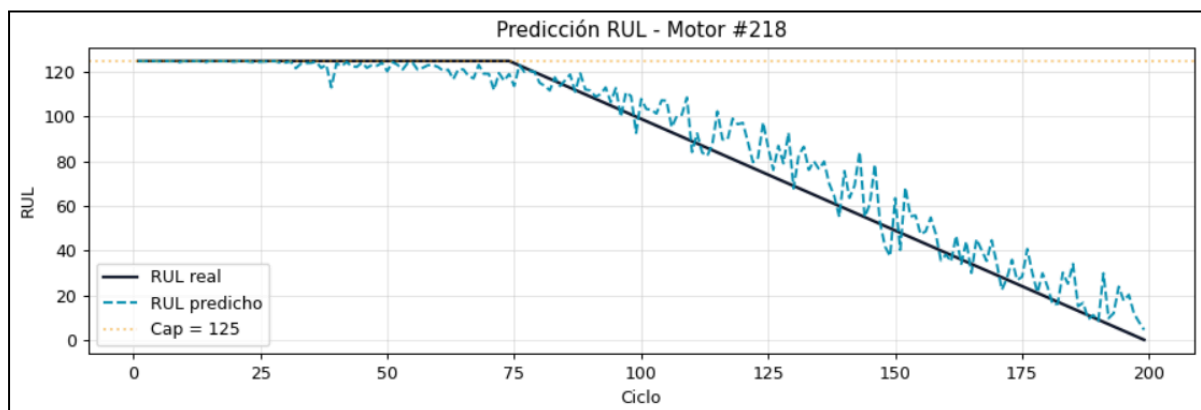
Viendo la evolución de las variables y sin utilizar ningún modelo, a simple vista la variable que más relación tiene con el desgaste del motor es la temperatura (**temperature\_mean**). Pero después veremos cómo, al utilizar un modelo predictivo, esta no será la variable más relevante relacionada con el desgaste del motor, sino que será **vibration\_std**.

## Modelos Evaluados y Resultados

Vamos a medir el rendimiento utilizando métrica del Error Absoluto Medio (MAE) y además, para elegir el mejor modelo predictivo hemos decidido probar dos estrategias distintas:

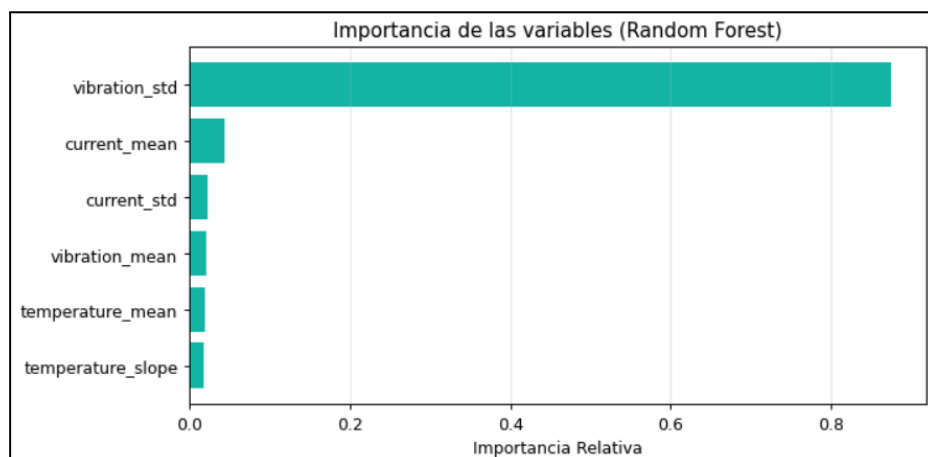
1. **Baseline (Regresión Lineal):** Un modelo base rápido y de baja complejidad. Ha obtenido un MAE global de **15.48** ciclos y un error en el tramo final (últimos 50 ciclos) de **14.60** ciclos.
2. **Modelo Random Forest:** Un ensamblaje no lineal de 100 árboles de decisión. Alcanza un MAE global de **10.06** ciclos y en los últimos 50 ciclos el error se reduce a **9.62** ciclos.

En base a este resultado, el mejor modelo ha sido el **Random Forest** por lo que este será el modelo principal usado para las pruebas. Además de la media general de ciclos hemos calculado la de los últimos 50 ciclos ya que es la fase más importante del motor, siendo **más importante** este resultado (9,52) que el mae global (10.06). Este es un ejemplo de la predicción del RUL, donde se puede ver como consigue acertar la predicción:



## Discusión de Características (Feature Importance)

Este es el resultado de las variables más importantes para el random forest:



Se puede observar como la intuición de las primeras gráficas que aparecían dando importancia a **temperature\_slope** sin utilizar ningún modelo predictivo eran engañosas, según nuestro modelo de Random Forest, el predictor dominante es **vibration\_std** con mas de un 80% de importancia y el segundo sería **current\_std**.

Desde el punto de vista del diseño de hardware (Edge AI), esta conclusión permitiría prescindir de los termopares en futuras iteraciones del sistema, abaratando el coste del dispositivo IoT sin comprometer la precisión predictiva.

### **Análisis Crítico y Mejoras**

**Trade-offs:** Elegimos el Random Forest frente al *baseline* lineal. Ganamos ~10 ciclos de precisión en la fase crítica, pero sacrificamos interpretabilidad directa (es un modelo más opaco).

**Limitaciones:** El modelo devuelve un RUL determinista y puntual. Carece de cuantificación de incertidumbre (intervalos de confianza) crucial en la industria. Además, evalúa los ciclos de forma aislada (ceguera temporal) y depende de un *capping* estático (125 ciclos) que introducirá sesgos si el régimen físico de la máquina cambia.

**Propuestas de Mejora:** Evolucionar a modelos recurrentes (LSTM/GRU) para capturar el desgaste histórico, e implementar un enfoque en cascada: primero clasificar si el motor está sano o degradado, y luego aplicar la regresión de RUL solo en la fase de degradación.

## T3 - ANÁLISIS DE DESPLIEGUE EDGE

### Perfilado y Cuantización

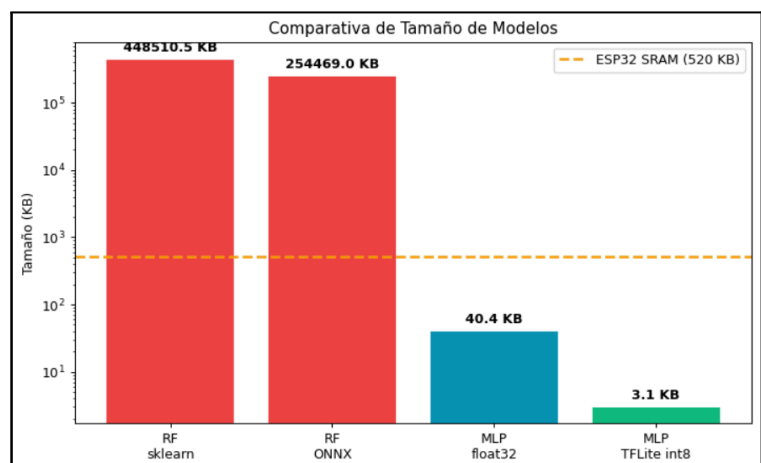
Para evaluar la viabilidad de trasladar el modelo predictivo desde un entorno Cloud hacia la planta industrial (Edge), se ha realizado un perfilado estricto de consumo de memoria y latencia. Se comparó el modelo **Random Forest** original frente al modelo **ONNX**, una red **MLP en float 32** y otra MLP cuantizada a **int8** mediante **TensorFlow Lite**.

Los resultados empíricos extraídos del entorno de pruebas se consolidan en la siguiente tabla resumen:

| Modelo          | Tamaño (KB) | Lat. PC (ms) | Lat. ESP32 (ms) | MAE Final (ciclos) | Cabe ESP32? |
|-----------------|-------------|--------------|-----------------|--------------------|-------------|
| RF sklearn      | 448510.5    | 31.3223      | 1566.11         | 9.62               | ✗ No        |
| RF ONNX         | 254469.0    | 0.0180       | 0.90            | 9.62               | ✗ No        |
| MLP float32     | 40.4        | —            | —               | 10.04              | ✓ Sí        |
| MLP TFLite int8 | 3.1         | 0.0172       | 0.22            | 28.24              | ✓ Sí        |

**Nota aclaratoria:** La latencia del modelo *MLP float32* se omite deliberadamente. Al medir inferencias unitarias, la sobrecarga interna de la API de Keras en PC genera tiempos artificialmente altos que no reflejan el coste matemático real, impidiendo una comparativa justa frente a formatos optimizados para producción (ONNX y TFLite).

Y aquí a la derecha tenemos una comparativa visual de los diferentes tamaños de cada modelo:



## Decisión de Despliegue y Justificación Arquitectónica

Tras perfilar los modelos, la decisión arquitectónica para producción es desplegar el sistema en un **Edge Server local (por ejemplo, un IPC industrial o Raspberry Pi)**, el mejor formato de modelo es el **RF ONNX**.

### ¿Por qué rechazar RF sklearn?

Aunque mantiene la precisión óptima (MAE 9.62), su tamaño en bruto es muy grande (448.5 MB) y su latencia de inferencia no está optimizada (31.3223 ms en PC). Mantener un modelo sin serializar en producción es ineficiente.

### ¿Por qué rechazar MLP float32?

Se trata de un modelo intermedio. Por un lado, pierde precisión frente al Random Forest (el MAE sube un poco, al 10.04), pero obtiene un resultado mucho mejor que TFLite, pero por otro lado, no es tan ligero como TFLite, aunque sí cabría en un ESP32.

### ¿Por qué rechazar MLP TFLite int8 en MCU?

Este modelo cabe físicamente en los 520 KB de SRAM de un microcontrolador tipo ESP32, ocupando apenas **3.1 KB**. Sin embargo, el *trade-off* es inaceptable para el negocio: la precisión en el tramo crítico empeora en **18.62 ciclos** (pasando de un MAE de 9.62 a 28.24). En la monitorización de maquinaria pesada, este margen de error implica un altísimo riesgo de avería catastrófica no detectada.

Por tanto, el modelo **RF ONNX** se consolida como la solución ideal para el Edge Server, apoyándose en la arquitectura de los 5 pilares fundamentales del Edge AI :

1. **Latencia:** El formato ONNX optimiza el grafo computacional, logrando tiempos de inferencia de apenas **0.0180 ms**, garantizando un tiempo de reacción inmediato.
2. **Ancho de Banda:** Filtrar y procesar la telemetría *on-premise* evita el envío constante de gigabytes de registros de alta frecuencia (vibración, corriente) a la nube.
3. **Disponibilidad (Offline):** El cálculo del RUL no se detendrá aunque la conectividad externa de la planta logística sufra caídas.
4. **Privacidad:** Los datos brutos de la maquinaria, que pueden revelar ritmos de producción y considerarse secreto industrial, nunca abandonan la red local.
5. **Costes:** Al disponer de alimentación eléctrica continua en el cuadro de mandos, no existe la restricción energética de una batería de un MCU. El coste del hardware local (Edge Server) se amortiza rápidamente al evitar las cuotas recurrentes de computación en un proveedor Cloud.

## Justificación de Despliegue y Limitaciones

**Trade-offs y Decisión Edge:** Se descarta el MCU (ESP32). Aunque el modelo cuantizado a int8 ocupa solo **3.1 KB**, la pérdida de **18.62 ciclos** de precisión es inasumible. La decisión es desplegar un Edge Server local (ej. Raspberry Pi) que soporta los **254.4 MB** del RF en ONNX, manteniendo el MAE óptimo (**9.62**).

**Razones de usar Edge AI:** Esta arquitectura garantiza latencia mínima (**0.0180 ms**), disponibilidad *offline*, y protege la privacidad del secreto industrial al no enviar gigabytes de telemetría bruta a la nube.

**Propuesta de Mejora:** El despliegue actual es estático. La evolución requiere implementar Federated Learning, permitiendo a cada Edge Server entrenar localmente con nuevas anomalías y enviar solo la actualización de pesos al servidor central.

# T4 - THREAT MODEL AIoT

## Resumen de Vectores de Ataque Analizados

A partir de las pruebas empíricas desarrolladas en el entorno de evaluación (ver *Anexo de Código T4*), se ha evaluado la viabilidad, el impacto y las defensas de los principales vectores de ataque contra nuestra arquitectura industrial predictiva:

| Vector                  | ¿Atacable en nuestro E2?                                                                                                                                                                          | Demostración Empírica                                                                                                                                                                         | Defensa Propuesta                                                                                   | Trade-off                                                                                                                                            |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sensor Spoofing         | Sí. Un atacante podría obtener acceso físico a los cuadros de los motores en la planta logística.                                                                                                 | Un ataque simple se detectó en un <b>18.0%</b> , pero el <i>spoofing</i> coordinado (manteniendo coherencia física) evadió el detector bajando la tasa a <b>0.0%</b> .                        | Inclusión de características temporales (desviación estándar móvil) y validación cruzada.           | Mayor consumo de memoria RAM en el Edge Server al almacenar ventanas temporales continuas.                                                           |
| Adversarial (FGSM)      | No. Nuestro modelo predictivo (T2) es un <i>Random Forest</i> . Al estar basado en árboles de decisión no diferenciables, carece de gradientes, haciéndolo matemáticamente inmune al ataque FGSM. | En pruebas aisladas (P13), el FGSM duplicó el error de una red neuronal. Este impacto justifica nuestra decisión arquitectónica de priorizar modelos basados en árboles para el entorno Edge. | Diseño por arquitectura: selección de algoritmos discretos frente a funciones de pérdida continuas. | El modelo RF no protege contra ataques adversariales de caja negra, limitando la seguridad frente a atacantes con acceso continuo a las inferencias. |
| Data Poisoning          | Sí. El Edge Server está configurado para ejecutar ciclos de reentrenamiento automáticos <i>on-premise</i> .                                                                                       | La inyección de un 5% de datos maliciosos en la ventana de fallo inminente empeoró el MAE crítico de <b>9.51 a 10.54</b> .                                                                    | Evaluación exclusiva de métricas estratificadas y firma de datos ( <i>Provenance</i> ).             | Requiere infraestructura de datos más compleja para mantener el linaje de cada lectura histórica.                                                    |
| Privacidad / Extracción | No. La arquitectura de la T3 es 100% local en un dispositivo Edge sin red federada.                                                                                                               | Teóricamente, en pruebas aisladas (P13), forzar Privacidad Diferencial ( $\epsilon = 10$ ) elevó el error a niveles inoperativos.                                                             | Aislamiento físico de la red local.                                                                 | Despliegue descentralizado que impide aprovechar el aprendizaje colaborativo entre distintas fábricas.                                               |

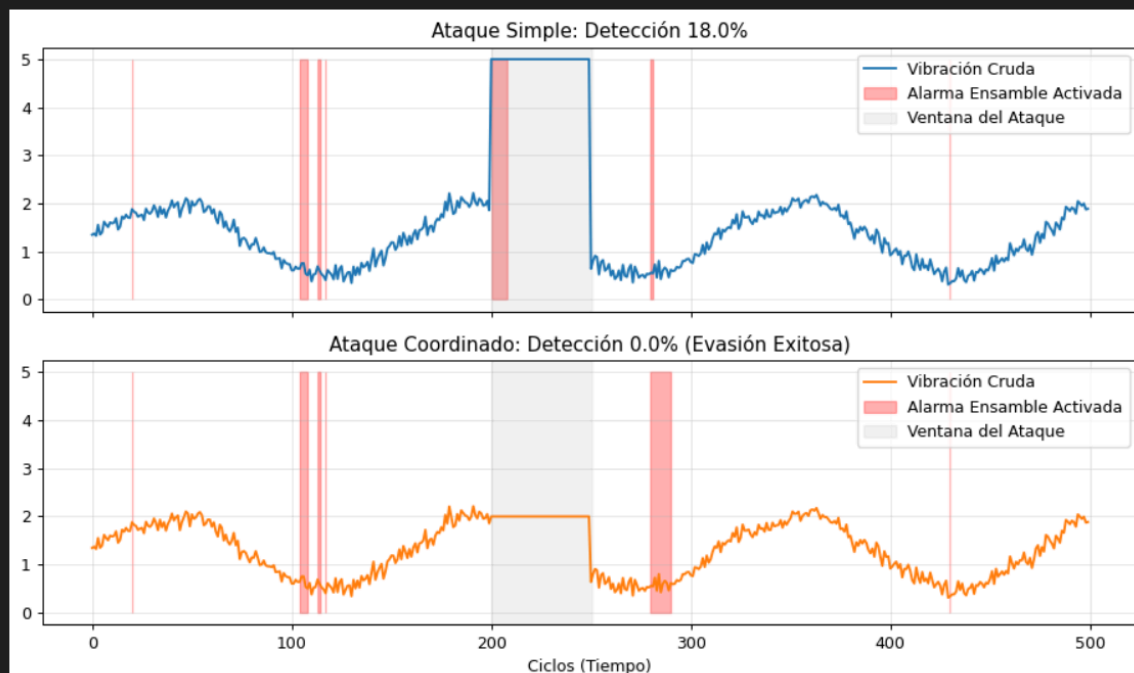
## Análisis Detallado de Vulnerabilidades Plausibles

De acuerdo con el análisis de la arquitectura, en el escenario de la entrega, se detallan a continuación los tres vectores de ataque viables, incluyendo afectaciones directas a los modelos de IA y la infraestructura.

### Vector 1: Sensor Spoofing Coordinado (Ataque al Modelo T1)

Visualización de diferencia entre un ataque simple y coordinado (el simple se detecta):

Detección de ataque simple (1 sensor irreal): 18.0%  
Detección de ataque sofisticado (3 sensores coherentes): 0.0%



1. **Descripción:** El atacante obtiene acceso físico a los cuadros de los motores en la planta logística y manipula la lectura de los sensores de forma coordinada. Altera simultáneamente la corriente, la vibración y la temperatura para simular una sobrecarga operativa coherente con la física de la máquina. Es viable debido a la frecuente falta de seguridad perimetral estricta alrededor de la maquinaria pesada en entornos industriales.
2. **Impacto:** Este ataque evade las defensas explotando la propia mecánica de adaptación al *Concept Drift* (desgaste natural) del modelo de la Tarea 1. Aunque el Ensemble es excelente adaptándose al deterioro gradual del motor gracias a su ventana temporal (Z-Score), esta misma característica es su talón de Aquiles frente a un ataque sostenido. En el ataque simple, el sistema detectó la anomalía inicial de forma deficiente (**18.0%** de detección). Sin embargo, en el ataque coordinado, al mantener el atacante una coherencia física artificial, la ventana móvil asimiló los datos maliciosos rápidamente, recalibró su media y aceptó el ataque como el "nuevo estado normal" (**0.0%** de detección). La decisión errónea derivada es no emitir la alerta correspondiente, provocando el fallo físico del motor y una parada de producción no programada.
3. **Contramedida razonada:**
  - 3.1. Defensa: Modificar el pipeline de extracción de características para incluir umbrales estáticos absolutos que no dependan exclusivamente de variables dinámicas, evitando así que el modelo normalice ataques continuados.
  - 3.2. Coste: Bajo, es una modificación exclusiva de software en la lógica del preprocesamiento de datos.
  - 3.3. Trade-off: Implica un mayor consumo de memoria RAM y tiempo de cómputo en el Edge Server.

## Vector 2: Data Poisoning "Targeted" (Ataque al Modelo T2):

```
--- IMPACTO DEL ATAQUE TARGETED ---  
MAE Global Limpio: 10.06 -> Envenenado: 10.30  
MAE Crítico Limpio: 9.51 -> Envenenado: 10.54
```

1. **Descripción:** Explotación de los reentrenamientos locales en el Edge Server. Mediante *malware* en la base de datos interna, el atacante altera de forma selectiva un 5% de las etiquetas históricas, marcando lecturas de fallo inminente ( $RUL < 25$ ) como estado sano ( $RUL = 100$ ). Es altamente viable porque el sistema Edge automatiza la ingesta de datos de la planta sin supervisión humana.
2. **Impacto:** El *Random Forest* de la Tarea 2 aprende a ignorar los síntomas previos a la rotura. Durante la prueba empírica, este envenenamiento mantuvo el ataque oculto a la monitorización estándar (el MAE global pasó de 10.06 a 10.30), pero degradó el MAE crítico de los últimos 50 ciclos de **9.51** a **10.54**. La decisión errónea es un retraso fatal en la estimación del RUL, impidiendo el mantenimiento a tiempo.
3. **Contramedida razonada:**
  - 3.1. Defensa: Implementar un sistema de procedencia de datos (*Provenance*) para descartar lecturas no verificadas.
  - 3.2. Coste: Alto. Requiere rediseñar el esquema de la base de datos temporal para incluir firmas digitales por cada lectura.
  - 3.3. Trade-off: Aumenta significativamente la complejidad de la infraestructura de almacenamiento local y el tamaño en disco de los registros.

## Vector 3: Man-in-the-Middle e Inyección de Ruido (Ataque a la Infraestructura)

1. **Descripción:** Interceptación del tráfico en la red de área local (LAN). El atacante se sitúa entre los sensores IoT de la maquinaria y el Edge Server, inyectando ruido matemático de alta frecuencia en el flujo de telemetría. Es viable porque las redes industriales a menudo transmiten datos utilizando protocolos en texto plano sin autenticación.
2. **Impacto:** El ruido inyectado desploma artificialmente la estimación de vida útil (RUL), provocando falsas alarmas repetitivas. La decisión errónea inducida es la programación de mantenimientos innecesarios y la sustitución prematura de componentes mecánicos sanos.
3. **Contramedida razonada:**
  - 3.1. Defensa: Sustituir los protocolos de transmisión en texto plano por MQTT sobre TLS (mTLS) con autenticación mutua mediante certificados *on-premise*. Además, añadir una validación de entradas que descarte alteraciones de frecuencia físicamente imposibles.
  - 3.2. Coste: Medio. Requiere la configuración y gestión de una Infraestructura de Clave Pública (PKI) interna para la fábrica.
  - 3.3. Trade-off: Incrementa la latencia en las lecturas y la carga de CPU en los microcontroladores debido al cifrado en tiempo real.

## Plan de Detección y Respuesta a Incidentes

Protocolo automatizado para responder al compromiso del modelo por **Data Poisoning Targeted**:

1. **Detección:** El sistema evalúa el MAE estratificado. Se levanta una alerta crítica si el MAE en la franja de los últimos 50 ciclos empeora más de un 10% tras un reentrenamiento local.
2. **Contención:** El Edge Server bloquea el despliegue del nuevo modelo a producción y ejecuta un *rollback* de emergencia a la versión estable anterior. Se detienen los reentrenamientos programados.
3. **Eradicación:** Mediante las firmas de *Provenance*, se aísla la fuente que introdujo las etiquetas anómalas y se purga su historial de la base de datos local.
4. **Recuperación:** Se recompila un conjunto de datos histórico validado y se lanza un entrenamiento en frío para restaurar el modelo predictivo.
5. **Lecciones Aprendidas:** Se incorpora la firma de este envenenamiento al sistema de monitorización estática y se actualiza el modelado de amenazas.

## Conexiones Explícitas con los Modelos y Limitaciones del Sistema

La seguridad del sistema se ha abordado condicionando la arquitectura de los modelos desarrollados previamente:

- **Conexión con T1 (Detector de Anomalías):** La defensa frente al vector de *Sensor Spoofing* invalida la eficacia de evaluar lecturas estáticas. Exige que el modelo final implementado procese obligatoriamente series temporales continuas para ser resistente a evasiones.
- **Conexión con T2 (Modelo Predictivo):** El riesgo crítico del *Data Poisoning* justifica operativamente nuestra decisión adoptada en la T2 de descartar el MAE global como métrica principal, forzando la evaluación continua del rendimiento empírico en la "ventana crítica". Además, la elección algorítmica de un *Random Forest* actúa como una defensa de diseño, proporcionando inmunidad nativa frente a ataques adversariales de gradiente (FGSM).
- **Conexión con T3 (Despliegue Edge):** El aislamiento en un nodo local (Edge) elimina riesgos de extracción mediante redes federadas, haciendo innecesaria la Privacidad Diferencial que degradaría la utilidad del modelo. Se asume como **limitación de diseño** que la restricción de cómputo en el Edge Server imposibilita el entrenamiento de modelos de defensa pesados.

## REFLEXIÓN TRANSVERSAL

El sistema diseñado a lo largo de este proyecto establece un *pipeline* funcional que abarca desde la monitorización reactiva de anomalías (T1) hasta la inferencia predictiva del RUL en el Edge (T2 y T3), operando bajo un modelo de amenazas definido (T4). Sin embargo, el despliegue actual presenta limitaciones estructurales inherentes a las arquitecturas centralizadas y estáticas. Para elevar este sistema a un estándar de Industria 4.0 maduro, la evolución natural exige la integración de **Federated Learning** y la adopción de un **Digital Twin**.

### ¿Dónde encajaría Federated Learning (FL) en nuestro sistema?

Actualmente, nuestra decisión de despliegue (T3) ubica el modelo predictivo (Random Forest en formato ONNX) de forma aislada en un Edge Server local para garantizar la privacidad del secreto industrial y minimizar la latencia. El problema de este aislamiento es que si el "Motor A" en una planta de Madrid experimenta un modo de fallo inédito, el modelo del "Motor B" en Barcelona jamás aprenderá de esa experiencia.

**Ubicación Arquitectónica y Sinergias:** El *Federated Learning* encajaría exactamente en la **fase de reentrenamiento y actualización de los Edge Servers**. En lugar de tener modelos estáticos, la arquitectura transicionaría a una red colaborativa:

- **Nodos Locales (Edge Servers):** Cada planta industrial retendría su telemetría cruda (vibración, corriente, temperatura) asegurando la privacidad absoluta. Al ocurrir un fallo real, el Edge Server calcularía el error de su predicción y ejecutaría un reentrenamiento local.
- **Agregador Central (Cloud):** Los Edge Servers enviarían periódicamente únicamente la actualización matemática de los pesos (o la estructura de sus árboles) a un servidor central. El Cloud agregaría este conocimiento y devolvería un modelo global unificado mucho más robusto.

**Desafíos Técnicos y Modificaciones Necesarias:** La inclusión de FL nos obligaría a replantear nuestra decisión de la Tarea 2. Nuestro modelo ganador actual es un *Random Forest*. A diferencia de las redes neuronales, los árboles de decisión no utilizan el descenso de gradiente (SGD), lo que dificulta usar el algoritmo estándar *FederatedAveraging* (*FedAvg*).

- **Solución:** Para integrar FL, podríamos cambiar la arquitectura a un ensamble de *Federated Forests* (donde el Cloud agrupa los árboles independientes creados por cada planta), o bien, recuperar nuestra red **MLP float32** evaluada en la T3. Aunque el MLP sacrificaba algo de precisión inicial (MAE de 10.04 frente al 9.62 del RF), su naturaleza diferenciable lo hace matemáticamente idóneo para el *Federated Learning*.

## Impacto en la Seguridad (T4) y Trade-offs:

- *Ventaja (Privacidad)*: Resuelve de raíz el vector de extracción de datos, ya que los *raw data* nunca cruzan la red LAN.
- *Trade-off (Vulnerabilidad)*: Amplía la superficie de ataque para el *Data Poisoning*. Si un atacante compromete un Edge Server y altera localmente el entrenamiento (como demostramos en la T4 con el aumento del MAE a 10.54), el nodo enviará pesos maliciosos al Cloud, pudiendo envenenar el modelo global de toda la corporación. Obligaría a desplegar defensas de *Byzantine Fault Tolerance* en el agregador.

## ¿Qué aportaría un Digital Twin (DT) al sistema?

Un *Digital Twin* es una réplica virtual y matemáticamente exacta del motor físico, alimentada en tiempo real por el flujo MQTT de nuestra arquitectura IoT. Su inclusión supondría el salto cualitativo más drástico del proyecto: pasar de un sistema puramente **Predictivo** (saber cuándo va a fallar) a uno **Prescriptivo** (saber qué hacer para evitar que falle).

## Aportaciones directas a los módulos desarrollados:

1. **Detección de Anomalías Basada en la Física (Mejora de la T1)**: Nuestro Ensamble actual (Z-Score + Isolation Forest) es robusto, pero es de naturaleza puramente estadística. Ignora la física de la máquina. Un DT simularía la termodinámica y el electromagnetismo del motor en tiempo real. En lugar de comparar la vibración actual contra los últimos 100 ciclos estadísticos, la compararíamos contra el *estado físico esperado* que dicta el DT bajo la carga de trabajo actual. Esto eliminaría la "ceguera adaptativa" frente a derivas extremadamente lentas, logrando un *Zero False Positive Rate* en entornos de alta variabilidad operativa.
2. **Simulación "What-If" para extender el RUL (Evolución de la T2)**: Si nuestro modelo *Random Forest* predice que al motor le quedan 25 ciclos de vida (RUL=25), actualmente el operario solo puede programar el reemplazo. Con un DT, el sistema podría simular de forma autónoma escenarios operativos futuros: "*¿Qué ocurre si reducimos la carga del motor al 80% y aumentamos la refrigeración ambiental?*". El DT calcularía el nuevo estrés físico e informaría de que, bajo esas nuevas condiciones, el RUL se extendería a 45 ciclos. Esto otorga al equipo de logística un margen de maniobra invaluable para que la pieza de repuesto llegue sin tener que detener la cadena de montaje.
3. **Generación de Datos Sintéticos y Seguridad (Apoyo a la T4)**: Entrenar modelos de detección de *Sensor Spoofing* es peligroso, ya que requiere destruir o comprometer maquinaria física para obtener los datos. El DT permite ejecutar simulaciones de ciberataques coordinados y fallos catastróficos en el entorno virtual. Esto generaría *datasets* sintéticos masivos y seguros para pre-entrenar al Ensamble de la T1 frente a vectores de ataque altamente sofisticados que nunca se han visto en el mundo real.

**Trade-offs del Digital Twin:** El beneficio operativo de un DT es incalculable, pero su coste arquitectónico es masivo.

- *Computación:* Las simulaciones de elementos finitos (FEA) requeridas por un DT son inviables en un dispositivo *Edge* restringido. Exigiría devolver gran parte de la carga computacional pesada a un *Cloud* o a un clúster local de alta computación, yendo en contra de la descentralización extrema propuesta en la T3.
- *Complejidad Multidisciplinar:* Ya no bastaría con ingenieros de *Machine Learning*. Requeriría la integración constante de ingenieros mecánicos para calibrar la réplica física, multiplicando los costes de mantenimiento del propio *software*.

### **Conclusión**

La arquitectura actual ha demostrado empíricamente ser una base sólida, eficiente y auditable. No obstante, el ecosistema industrial del futuro requerirá **Federated Learning** para transformar los nodos aislados en un cerebro logístico colectivo de forma privada, y dependerá del **Digital Twin** para romper el techo de cristal de la Inteligencia Artificial: dejar de limitarse a informar sobre el futuro para empezar a modificarlo activamente mediante decisiones prescriptivas simuladas en tiempo real.

## CONCLUSIÓN FINAL

En el presente proyecto se ha diseñado, implementado y evaluado exhaustivamente una arquitectura completa de Inteligencia Artificial de las Cosas (AloT) orientada al Mantenimiento Predictivo Industrial. La aproximación metodológica adoptada ha permitido ir más allá de la simple aplicación de algoritmos aislados, consolidando un *pipeline* integral que resuelve problemas reales del entorno de fabricación.

Se ha logrado superar la barrera del desgaste natural de la maquinaria (*Concept Drift*) mediante el diseño de un detector de anomalías en Ensamble (T1), se ha transicionado de un enfoque reactivo a uno proactivo mediante la regresión del RUL con un modelo *Random Forest* (T2), se ha optimizado la huella computacional y la privacidad desplegando el sistema en formato ONNX sobre un servidor Edge local (T3), y se han acotado los riesgos operativos mediante la definición de un *Threat Model* (T4).

**Limitaciones Estructurales del Sistema** A pesar de la robustez empírica demostrada por la arquitectura, la evaluación crítica del ciclo de vida del sistema revela las siguientes limitaciones inherentes a nuestras decisiones de diseño:

1. **Determinismo Predictivo (Carencia de Incertidumbre):** Nuestro modelo predictivo principal (T2) devuelve un RUL puntual y absoluto. En la industria crítica, carecer de intervalos de confianza o de una cuantificación de la incertidumbre (incertidumbre epistémica y aleatoria) limita la capacidad del equipo logístico para evaluar el riesgo real de estirar la vida útil de un componente.
2. **Ceguera Temporal del Regresor:** Aunque nuestro detector de anomalías procesa ventanas temporales (T1), el modelo de *Random Forest* (T2) evalúa características estáticas en cada instante de tiempo de forma independiente. Esta incapacidad para capturar relaciones secuenciales complejas a largo plazo restringe el rendimiento del modelo frente a patrones de degradación no lineales.
3. **Aislamiento del Conocimiento (Falta de FL):** El despliegue estático 100% *on-premise* en un Edge Server (T3), aunque blinda la privacidad, impide que el sistema aproveche el aprendizaje colaborativo. Las anomalías descubiertas por este motor no sirven para mejorar la detección en motores de otras plantas.
4. **Vulnerabilidad de los Reentrenamientos Automatizados:** Como demostró el análisis de ciberseguridad (T4), la dependencia de ciclos de reentrenamiento ciegos en el Edge abre una brecha crítica para ataques de *Data Poisoning* local, requiriendo infraestructuras secundarias pesadas (como firmas *Provenance*) para su mitigación.

**Conclusión Final** El diseño de sistemas AloT de grado de producción no consiste en buscar un algoritmo infalible, sino en gestionar de forma consciente, medible y auditable los rigurosos *trade-offs* entre precisión matemática, latencia computacional, consumo de recursos y ciberseguridad. La arquitectura propuesta en esta memoria demuestra que, aplicando un criterio ingenieril sólido, es posible construir ecosistemas inteligentes en el Edge capaces de asimilar la variabilidad del mundo físico, protegiendo tanto la integridad mecánica de la maquinaria como el secreto industrial de la planta operativa.